

```
!python -m spacy download en_core_web_sm
```

Collecting en-core-web-sm==3.8.0

Using cached https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl
 ✓ Download and installation successful

You can now load the package via `spacy.load('en_core_web_sm')`

⚠ Restart to reload dependencies

If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.

```
import spacy
```

```
nlp = spacy.load("en_core_web_sm")
```

```
text = "Apple is looking at buying U.K. startup for $1 billion"
```

```
doc = nlp(text)
```

```
print("Named Entities, Phrases and concepts")
```

```
for ent in doc.ents:
```

```
    print(f"{ent.text:15} - {ent.label_:10} {ent.start_char:10} {ent.end_char:10}")
```

```
Named Entities, Phrases and concepts
```

Apple	- ORG	0	5
U.K.	- GPE	27	31
\$1 billion	- MONEY	44	54

```
import spacy
```

```
nlp = spacy.load("en_core_web_sm")
```

```
doc = nlp("data science and AI & gen AI has great career a head")
```

```
for token in doc:
```

```
    print(token.text)
```

```
data
science
and
AI
&
gen
AI
has
great
career
a
head
```

```
for token in doc:
```

```
    print(token.pos_)
```

```
NOUN
NOUN
CCONJ
PROPN
CCONJ
PROPN
PROPN
VERB
ADJ
NOUN
DET
NOUN
```

```
for token in doc:
```

```
    print(token.text, ': ', token.pos_)
```

```
data : NOUN
science : NOUN
and : CCONJ
AI : PROPN
& : CCONJ
gen : PROPN
AI : PROPN
has : VERB
great : ADJ
career : NOUN
a : DET
head : NOUN
```

```
for token in doc:
    print(token.text, ':', token.pos_, '-->', token.lemma, token.dep_)
```

```
data : NOUN --> 6645506661261177361 compound
since : NOUN --> 1385684616247317504 nsubj
and : CONJ --> 2283656566040971221 cc
AI : PROPN --> 5530044837203964789 conj
& : CONJ --> 15473034735919704609 cc
gen : PROPN --> 11090586419576673076 compound
AI : PROPN --> 5530044837203964789 conj
has : VERB --> 14692702688101715474 ROOT
great : ADJ --> 8881679497796027013 amod
career : NOUN --> 16267834800013645067 dobj
a : DET --> 11901859001352538922 det
head : NOUN --> 8419699711262724776 dobj
```

```
doc = nlp("Apple is looking at buying U.K. startup for $1 billion")
```

```
for token in doc:
    print(token.text, token.lemma, token.pos_, token.tag_, token.dep_, token.shape_, token.is_alpha, token.is_stop)
```

```
Apple 6418411030699964375 PROPN NNP nsubj Xxxxx True False
is 10382539506755952630 AUX VBZ aux xx True True
looking 16096726548953279178 VERB VBG ROOT xxxx True False
at 11667289587015813222 ADP IN prep xx True True
buying 9457496526477982497 VERB VBG pcomp xxxx True False
U.K. 14409890634315022856 PROPN NNP nsubj X.X. False False
startup 7622488711881293715 VERB VBD ccomp xxxx True False
for 16037325823156266367 ADP IN prep xxx True True
$ 11283501755624150392 SYM $ quantmod $ False False
1 5533571732986600803 NUM CD compound d False False
billion 1231493654637052630 NUM CD pobj xxxx True False
```

```
# Text Summarization
```

text = ""
There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on. An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a given document. Image collection summarization is another application example of automatic summarization. It consists in selecting a representative image from a collection of images.

```
import spacy
from spacy.lang.en.stop_words import STOP_WORDS
from string import punctuation
```

```
stopwords = list(STOP_WORDS)
stopwords
```

```
['much',
 'they',
 'may',
 'd',
 'at',
 'do',
 'seem',
 'besides',
 'front',
 'otherwise',
 'empty',
 'along',
 'been',
 'side',
 'again',
 'rather',
 'in',
 'whole',
 'sometime',
 'alone',
 'someone',
 'did',
 'call',
 'thereby',
 'else',
 'the',
 'next',
 'four',
 'if',
 'over',
 'nine',
 'least',
 'being',
 'indeed',
 'into',
```

```
'anything',
'n't',
'part',
'somehow',
've',
'eleven',
'give',
'however',
'how',
'onto',
'whose',
'doing',
'd',
'full',
'beyond',
'get',
'amount',
'as',
'unless',
'elsewhere',
'go',
'put',
'during'
```

```
len(stopwords)
```

```
326
```

```
nlp = spacy.load('en_core_web_sm')
```

```
text
```

'There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on. The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether documents, or sets of images, or videos, news stories etc.). The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to a query. Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries depending on what the user needs.\nAn example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a given document. Sometimes one might be interested in generating a summary from a single source document, while others can use multiple source documents (for example, a cluster of articles on the same topic). This problem is called multi-document summarization. A related application is summar...

```
doc = nlp(text)
doc
```

There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on. The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether documents, or sets of images, or videos, news stories etc.). The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to a query. Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries depending on what the user needs.

An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a given document. Sometimes one might be interested in generating a summary from a single source document, while others can use multiple source documents (for example, a cluster of articles on the same topic). This problem is called multi-document summarization. A related application is summarizing news articles. Imagine a system, which automatically pulls together news articles on a given topic (from the web), and concisely represents the latest news as a summary.

Image collection summarization is another application example of automatic summarization. It consists in selecting a representative set of images from a larger set of images.[4] A summary in this context is useful to show the most representative images of results in an image collection exploration system. Video summarization is a related domain, where the system automatically creates a trailer of a long video. This also has applications in consumer or personal videos, where one might want to skip the boring or repetitive actions. Similarly, in surveillance videos, one would want to extract important and suspicious activity, while ignoring all the boring and redundant frames captured

```
# Tokens from text
tokens = [token.text for token in doc]
print(tokens)
```

```
['There', 'are', 'broadly', 'two', 'types', 'of', 'extractive', 'summarization', 'tasks', 'depending', 'on', 'what', 'the', 'su
```

```
doc
```

There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on. The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether documents, or sets of images, or videos, news stories etc.). The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to a query. Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries depending on what the user needs.

An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a given document. Sometimes one might be interested in generating a summary from a single source document, while others can use multiple source documents (for example, a cluster of articles on the same topic). This problem is called multi-document summarization. A related application is summarizing news articles. Imagine a system, which automatically pulls together news articles on a given topic (from the web), and concisely represents the latest news as a summary.

Image collection summarization is another application example of automatic summarization. It consists in selecting a representative set of images from a larger set of images.[4] A summary in this context is useful to show the most representative images of results in an image collection exploration system. Video summarization is a related domain, where the system automatically creates a trailer of a long video. This also has applications in consumer or personal videos, where one

might want to skip the boring or repetitive actions. Similarly, in surveillance videos, one would want to extract important and suspicious activity, while ignoring all the boring and redundant frames captured

```
# we have to calculate the frequency of each and every token, how many times repetition in text
word_frequencies = {}

for word in doc:
    if word.text.lower() not in stopwords:
        if word.text.lower() not in punctuation:
            if word.text not in word_frequencies.keys():
                word_frequencies[word.text] = 1
            else:
                word_frequencies[word.text] += 1
```

word_frequencies

```
'given': 2,
'interested': 1,
'generating': 1,
'single': 1,
'source': 2,
'use': 1,
'multiple': 1,
'cluster': 1,
'articles': 3,
'topic': 2,
'multi': 1,
'related': 2,
'application': 2,
'summarizing': 1,
'Imagine': 1,
'system': 3,
'pulls': 1,
'web': 1,
'concisely': 1,
'represents': 1,
'latest': 1,
'Image': 1,
'automatic': 1,
'consists': 1,
'selecting': 1,
'representative': 2,
'set': 2,
'larger': 1,
'images.[4': 1,
'context': 1,
'useful': 1,
'results': 1,
'image': 1,
'exploration': 1,
'Video': 1,
'domain': 1,
'creates': 1,
'trailer': 1,
'long': 1,
'video': 1,
'applications': 1,
'consumer': 1,
'personal': 1,
'want': 2,
'skip': 1,
'boring': 2,
'repetitive': 1,
'actions': 1,
'Similarly': 1,
'surveillance': 1,
'extract': 1,
'important': 1,
'suspicious': 1,
'activity': 1,
'ignoring': 1,
'redundant': 1,
'frames': 1,
'captured': 1}
```

len(word_frequencies)

103

```
max_frequency = max(word_frequencies.values())
max_frequency
```

11

```
# to get normalized/weighted frequency you should divide all frequencies with 11
```

```
for word in word_frequencies.keys():
    word_frequencies[word] = word_frequencies[word]/max_frequency
```

```
word_frequencies
```

```
{'given': 0.18181818181818182,
'interested': 0.09090909090909091,
'generating': 0.09090909090909091,
'single': 0.09090909090909091,
'source': 0.18181818181818182,
'use': 0.09090909090909091,
'multiple': 0.09090909090909091,
'cluster': 0.09090909090909091,
'articles': 0.2727272727272727,
'topic': 0.18181818181818182,
'multi': 0.09090909090909091,
'related': 0.18181818181818182,
'application': 0.18181818181818182,
'summarizing': 0.09090909090909091,
'Imagine': 0.09090909090909091,
'system': 0.2727272727272727,
'pulls': 0.09090909090909091,
'web': 0.09090909090909091,
'concisely': 0.09090909090909091,
'represents': 0.09090909090909091,
'latest': 0.09090909090909091,
'Image': 0.09090909090909091,
'automatic': 0.09090909090909091,
'consists': 0.09090909090909091,
'selecting': 0.09090909090909091,
'representative': 0.18181818181818182,
'set': 0.18181818181818182,
'larger': 0.09090909090909091,
'images.[4': 0.09090909090909091,
'context': 0.09090909090909091,
'useful': 0.09090909090909091,
'results': 0.09090909090909091,
'image': 0.09090909090909091,
'exploration': 0.09090909090909091,
'Video': 0.09090909090909091,
'domain': 0.09090909090909091,
'creates': 0.09090909090909091,
'trailer': 0.09090909090909091,
'long': 0.09090909090909091,
'video': 0.09090909090909091,
'applications': 0.09090909090909091,
'consumer': 0.09090909090909091,
'personal': 0.09090909090909091,
'want': 0.18181818181818182,
'skip': 0.09090909090909091,
'boring': 0.18181818181818182,
'repetitive': 0.09090909090909091,
'actions': 0.09090909090909091,
'Similarly': 0.09090909090909091,
'surveillance': 0.09090909090909091,
'extract': 0.09090909090909091,
'important': 0.09090909090909091,
'suspicious': 0.09090909090909091,
'activity': 0.09090909090909091,
'ignoring': 0.09090909090909091,
'redundant': 0.09090909090909091,
'frames': 0.09090909090909091,
'captured': 0.09090909090909091}
```

```
sentence_tokens = [sent for sent in doc.sents]
print(sentence_tokens)
```

[There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on., The first, An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a given document., Image collection summarization is another application example of automatic summarization., It consists in selecting a representative sentence from a collection of sentences.]

```
# we are going to calculate the sentence score
```

```
sentence_score = {}
```

```
for sent in sentence_tokens:
    for word in sent:
        if word.text.lower() in word_frequencies.keys():
            if sent not in sentence_score.keys():
                sentence_score[sent] = word_frequencies[word.text.lower()]
            else:
                sentence_score[sent] += word_frequencies[word.text.lower()]
```

sentence_score

```
{There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on.:
2.818181818181818,
The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether
documents, or sets of images, or videos, news stories etc.): 3.9999999999999987,
The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to
a query.: 3.909090909090909,
Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries depending
on what the user needs.: 3.2727272727272716,
An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a
given document.: 3.9999999999999996,
Sometimes one might be interested in generating a summary from a single source document, while others can use multiple source
documents (for example, a cluster of articles on the same topic): 2.545454545454545,
This problem is called multi-document summarization.: 1.8181818181818183,
A related application is summarizing news articles.: 1.0909090909090908,
Imagine a system, which automatically pulls together news articles on a given topic (from the web), and concisely represents
the latest news as a summary.: 2.9090909090909087,
Image collection summarization is another application example of automatic summarization.: 2.909090909090909,
It consists in selecting a representative set of images from a larger set of images.[4]: 1.1818181818181817,
A summary in this context is useful to show the most representative images of results in an image collection exploration
system.: 1.818181818181818,
Video summarization is a related domain, where the system automatically creates a trailer of a long video.:
2.2727272727272725,
This also has applications in consumer or personal videos, where one might want to skip the boring or repetitive actions.:
1.1818181818181817,
Similarly, in surveillance videos, one would want to extract important and suspicious activity, while ignoring all the boring
and redundant frames captured: 1.4545454545454544}
```

len(sentence_score)

15

from heapq import nlargest

```
select_length = int(len(sentence_tokens)*0.4)
select_length
```

6

```
# we have to select max 40% of all sentences
summary = nlargest(select_length, sentence_score, key = sentence_score.get)
summary
```

```
[An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a
given document.,
The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether
documents, or sets of images, or videos, news stories etc.),
The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to
a query.,
Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries depending
on what the user needs.,
Image collection summarization is another application example of automatic summarization.,
Imagine a system, which automatically pulls together news articles on a given topic (from the web), and concisely represents
the latest news as a summary.]
```

```
# If i need to combine these top 3 sentences then
final_summary = [word.text for word in summary]
final_summary
```

```
['An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a
given document.',
'The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether
documents, or sets of images, or videos, news stories etc.)',
'The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to
a query.',
'Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries
depending on what the user needs.\n',
'Image collection summarization is another application example of automatic summarization.',
'Imagine a system, which automatically pulls together news articles on a given topic (from the web), and concisely represents
the latest news as a summary.\n']
```

Start coding or [generate](#) with AI.

